# Scanner Identification Using Sensor Pattern Noise

**Group 28**

Course Instructor: Dr. Nitin Khanna
Course TA: Dheeraj Biswas


Group Members

Prince - 12341640
Pratik Raj - 12341620
Ayush Kumar - 12340440

**Abstract**

This report presents a scanner source identification pipeline based on Sensor Pattern Noise (SPN) extraction and supervised classification. The work begins with a TIFF dataset containing scanner-specific folders and multiple DPI variants. Only the 1200 DPI subset is used for the main experiment, while 200 DPI images and unwanted scanner folders are excluded.

Each valid image is partitioned into non-overlapping $1024 \times 768$ patches. After patch extraction, a 4-level Daubechies-8 wavelet denoising stage is used to obtain a noise residual for every patch. The residuals are then compressed into a 16-dimensional feature vector that captures both row-pattern statistics and correlation statistics. These features are used to train and evaluate an SVM classifier with group-aware cross-validation.

The final system achieves a best test accuracy of 88.44%, which is significantly better than the correlation-based baselines.

## Table of Contents

## Contents

# 1 Introduction

Scanner source identification is a digital forensics problem in which the goal is to determine which physical scanner produced a scanned image. Unlike generic image classification, this task depends on device-specific artifacts embedded in the output of the scanner hardware. These artifacts arise from stable manufacturing imperfections, sensor non-uniformity, and small systematic distortions in the readout process.

A common assumption in image forensics is that a device leaves a repeatable noise fingerprint. In scanners, this fingerprint is often expressed through fixed-pattern behavior along the CCD readout direction. If enough patches are aggregated, the random scene content averages out while the scanner-specific signature remains visible. This is the core idea behind SPN-based source identification.

The notebook workflow used for this report follows this logic: dataset traversal, patch extraction, wavelet-based denoising, noise residual analysis, feature engineering, and classifier training. The final SVM model is trained on a 16-dimensional representation derived from each patch, and the best configuration is obtained through group-aware cross-validated grid search. The same notebook also reports the 24,270 extracted patches and the final accuracy of 88.44%. :contentReference[oaicite:0]index=0

**Definition** (Sensor Pattern Noise)**.** *Given a scanned image $I$ and a denoiser $F(\cdot)$, the sensor pattern noise residual is defined as*

$$W = I - F(I).$$

*The residual $W$ is expected to suppress most scene content while preserving device-specific structure.*

## 1.1 Objectives

The report is organized around the following objectives:

1. Describe the scanner dataset and the selection of 1200 DPI images.
2. Explain the non-overlapping patch extraction procedure.
3. Present the wavelet-based SPN extraction method in detail.
4. Construct a compact 16-feature vector for each patch.
5. Train an SVM classifier and compare it with correlation baselines.
6. Interpret the results using feature plots and the confusion matrix.

## 1.2 Why this problem is difficult

The hardest part of the task is not simply classifying scanner models. In the dataset, there are two separate HP 6300c units. They are extremely similar at a hardware level, so the classifier must exploit very subtle residual differences rather than obvious visual cues. This is why the confusion matrix shows a much stronger distinction for the clearly different scanners and more overlap between the two 6300c units.

# 2 Dataset

## 2.1 Dataset Structure

The dataset is arranged as a hierarchy of scanner folders and DPI folders. The notebook reference shows that the complete dataset contains 1,583 TIFF files, but the main experiment uses only the 1200 DPI subset and only four scanner classes. The remaining 200 DPI images and additional scanner folders are excluded before feature extraction. This filtering is important because it makes the downstream classification problem consistent in terms of

image scale and scanner identity. :contentReference[oaicite:1]index=1

The final experimental set contains:

- 4 scanner classes,
- 160 images in total,
- 40 images per scanner class,
- 24,270 non-overlapping patches overall.

## 2.2 Scanner Classes

**Table 1:** Scanner classes used in the main experiment.

| ID | Scanner Model | Images | Approx. Patches |
|---|---|---|---|
| s1 | Epson 4490 | 40 | $\sim$ 6068 |
| s2 | HP ScanJet 6300c Unit 1 | 40 | $\sim$ 6068 |
| s3 | HP ScanJet 6300c Unit 2 | 40 | $\sim$ 6068 |
| s4 | HP ScanJet 8250 | 40 | $\sim$ 6066 |
| | **Total** | **160** | **24,270** |

## 2.3 A raw TIFF example

This figure is placed here so that the raw TIFF image appears early in the report, before the preprocessing discussion. It acts as the visual anchor for the dataset section.


**Figure 1:** One TIFF image from the dataset.

## 2.4 Observed Resolution Diversity

The notebook also reports that the full dataset spans many distinct image-size combinations, which is typical for scanner TIFF collections because different documents and source pages lead to different final raster dimensions. For the main pipeline, however, the image-size variation is not a blocking problem because the patch extraction step simply skips images smaller than one patch and otherwise extracts every valid non-overlapping window. This makes the pipeline robust to a heterogeneous scan collection. :contentReference[oaicite:2]index=2

# 3 Preprocessing Pipeline

## 3.1 A sample input preview

This second image is deliberately placed in a different section from the TIFF example so the two visuals are separated in the PDF while still supporting the same narrative about the raw inputs.

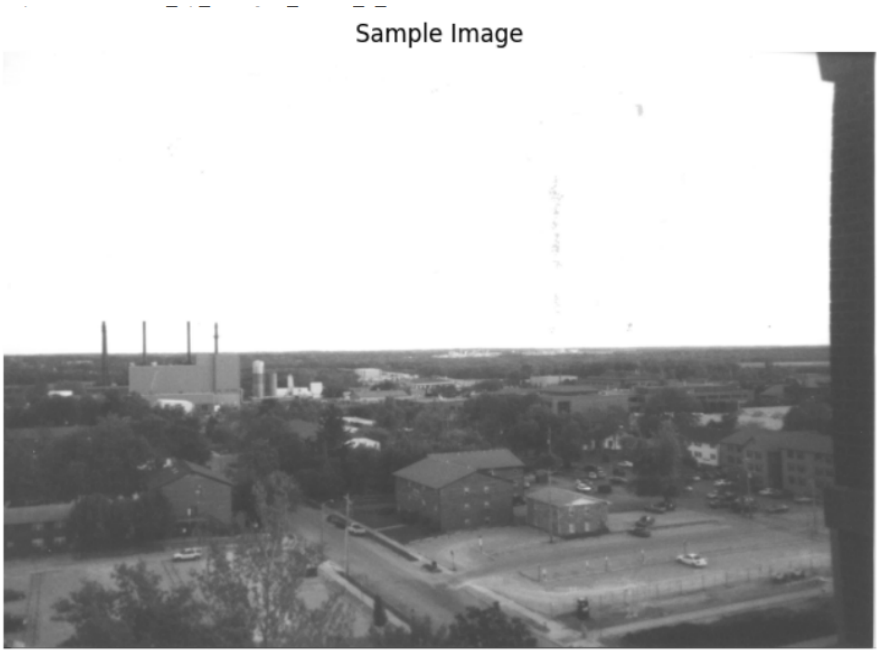
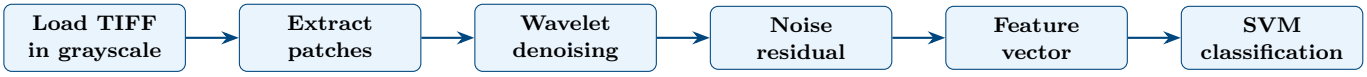



**Figure 2:** Sample image used to verify grayscale loading and visualization.

## 3.2 Overview

The preprocessing pipeline is deliberately simple and reproducible:




The key design choice is to keep the preprocessing deterministic. There is no random augmentation at the patch level, no synthetic transformation, and no content-based cropping. This makes the extracted residuals easier to compare across classes and reduces the chance that the classifier learns scene content instead of scanner signature.

## 3.3 Patch Extraction

Each image is divided into non-overlapping $1024 \times 768$ patches. The stride is equal to the patch size, so neighboring patches do not overlap. This is a structured form of sampling: it covers the available image area without creating redundant windows.

The notebook implementation shows that the patching routine first checks whether the image is large enough and then iterates across the image with the chosen stride. Images smaller than one full patch are skipped. This avoids partially padded patches that could introduce inconsistent border artifacts. :contentReference[oaicite:3]index=3

```
def extract_patches(image, patch_size=(1024,768), stride=(1024,768)):
    patches = []
    h, w = image.shape
    ph, pw = patch_size
    if h < ph or w < pw:
        return patches
    for i in range(0, h-ph+1, stride[0]):
```

```
        for j in range(0, w-pw+1, stride[1]):
            patch = image[i:i+ph, j:j+pw]
            if patch.shape == (ph,pw):
                patches.append(patch)
    return patches
```

**Listing 1:** Patch extraction used in the pipeline.

Because the patches are non-overlapping, the final patch count is determined by the image dimensions. The notebook reports exactly 24,270 extracted patches, which matches the summary used throughout this report. :contentReference[oaicite:4]index=4

### 3.4 Wavelet-Based Noise Extraction

The denoising stage uses a 4-level Daubechies-8 wavelet decomposition. In each patch, the finest-scale detail coefficients are used to estimate the noise standard deviation through the median absolute deviation estimator. The noise estimate is then used to compute a soft-threshold for the wavelet coefficients. After thresholding, the inverse transform reconstructs the denoised patch, and the residual is obtained by subtracting the denoised patch from the original patch.

$$\hat{\sigma} = \frac{\text{median}(|D_1|)}{0.6745} \tag{1}$$

$$\lambda_k = \hat{\sigma}\sqrt{2\ln n_k} \tag{2}$$

$$\hat{d} = \text{sgn}(d)\max(|d| - \lambda_k, 0) \tag{3}$$

$$\mathbf{N} = \mathbf{P} - \hat{\mathbf{P}} \tag{4}$$

```
import pywt
import numpy as np

coeffs = pywt.wavedec2(
    patch.astype(np.float32),
    'db8',
    level=4
)

sigma = np.median(np.abs(coeffs[-1][0])) / 0.6745

coeffs[1:] = [
    tuple(
        pywt.threshold(
            d,
            sigma*np.sqrt(2*np.log(d.size)),
            'soft'
        ) for d in lv
    )
    for lv in coeffs[1:]
]

denoised = pywt.waverec2(
    coeffs,
    'db8'
)[:patch.shape[0], :patch.shape[1]]

noise = patch.astype(np.float32) - denoised.astype(np.float32)
```

**Listing 2:** Wavelet denoising and residual extraction.

### 3.5 Preprocessing Output

The notebook shows a preview figure with the patch, the denoised patch, and the noise image side by side. The residual appears visually much flatter than the original image, which is exactly what is desired at this stage. The same notebook run also reports the final processing summary: 24,270 estimated patches, 24,270 processed noise samples, and successful completion of the entire loop. :contentReference[oaicite:5]index=5


**Figure 3:** Patch image, denoised image, and noise residual.

## 4 Noise Pattern Analysis

### 4.1 Row Noise Pattern

Once the residual is available, the next step is to summarize it into a compact representation that preserves scanner-specific structure. The notebook computes a row pattern by averaging the residual across rows, which produces a column-wise vector of length equal to the patch width. In practice, this vector acts like a scanner signature profile, because persistent column-wise bias is retained while unstructured image content is reduced.

$$r_j = \frac{1}{H} \sum_{i=1}^{H} N_{ij}, \qquad j = 1, \ldots, W \tag{5}$$

The corresponding plot in the notebook shows the row noise pattern against column index. Even though the curve looks noisy, the important point is that its statistical structure is repeatable across patches from the same scanner and is different enough across scanners to be useful for classification. :contentReference[oaicite:6]index=6

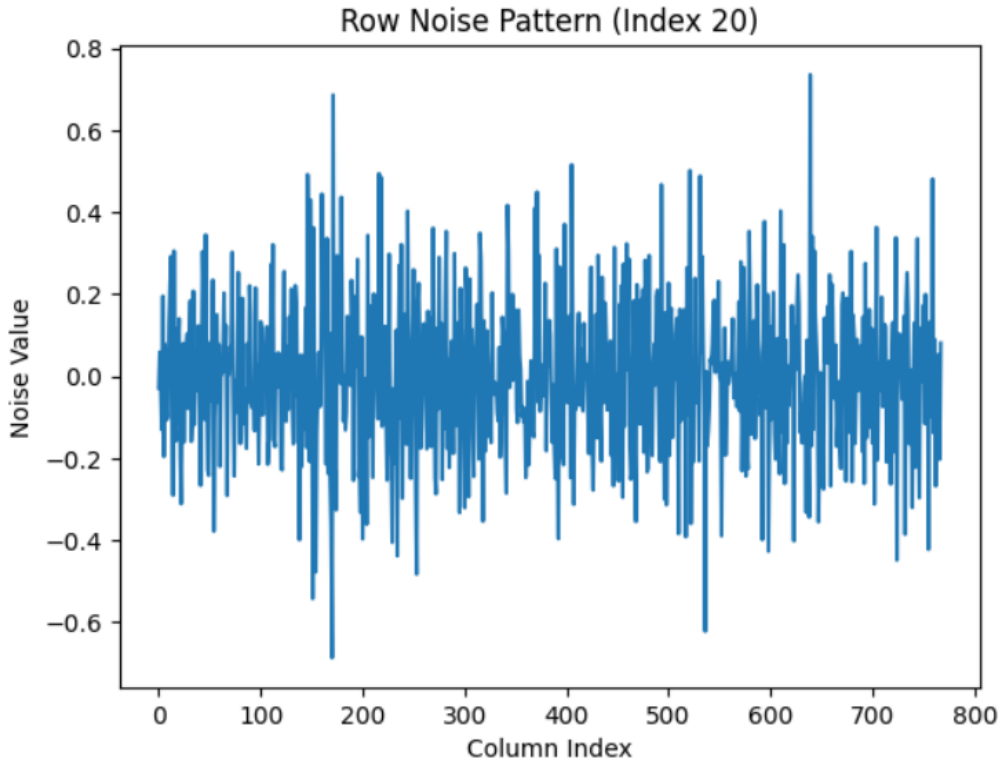



**Figure 4:** Row noise pattern: column index versus noise intensity.

### 4.2 Why the row pattern is useful

The row pattern works because scanner noise is not perfectly random. Some columns are consistently affected by the sensor readout chain, optics, or mechanical transport. Averaging across rows suppresses much of the scene structure and makes those column-wise effects easier to detect statistically.

### 4.3 Visual inspection of the residuals

The notebook's preview plots suggest that the extracted residual is not a clean binary separation of signal and noise. Instead, it is a weak fingerprint that must be amplified statistically. That is why the report uses sixteen features: the classifier benefits from multiple summaries of the same fingerprint rather than relying on a single scalar score. :contentReference[oaicite:7]index=7

## 5 Feature Extraction

### 5.1 Feature Design Philosophy

The goal of feature engineering here is to convert each patch-level residual into a small, fixed-length vector that is expressive enough for classification but small enough to avoid overfitting. The notebook implements this by extracting eight statistics from the row pattern and eight more statistics from the set of row-wise correlations. The result is a 16-dimensional feature vector.

### 5.2 Row Pattern Statistics

The first group of features is computed directly from the row pattern vector:

$$F_1 = \text{mean}(r), \qquad F_2 = \text{median}(r), \qquad F_3 = \text{std}(r), \tag{6}$$

$$F_4 = \text{skew}(r), \qquad F_5 = \text{kurtosis}(r), \qquad F_6 = \min(r), \tag{7}$$

$$F_7 = \max(r), \qquad F_8 = \sum_j r_j^2 \tag{8}$$

### 5.3 Correlation-Based Features

The second group of features is based on the Pearson correlation between each row of the residual and the row pattern vector.

$$\rho_i = \frac{\mathrm{cov}(\mathbf{n}_i, \mathbf{r})}{\sigma_{\mathbf{n}_i}\sigma_{\mathbf{r}}}.$$

Rows with near-zero variance are skipped to avoid numerical instability. Any non-finite correlation values are discarded before the summary statistics are computed.

$$F_9 = \mathrm{mean}(\rho), \qquad F_{10} = \mathrm{median}(\rho), \qquad F_{11} = \mathrm{std}(\rho), \tag{9}$$

$$F_{12} = \mathrm{skew}(\rho), \qquad F_{13} = \mathrm{kurtosis}(\rho), \qquad F_{14} = \min(\rho), \tag{10}$$

$$F_{15} = \max(\rho), \qquad F_{16} = \sum_i \rho_i^2 \tag{11}$$

### 5.4 Complete 16-D Feature Vector

The full feature vector is

$$\mathbf{x} = [F_1, F_2, \ldots, F_{16}]^\top.$$

The notebook reports that all 24,270 samples produce valid feature vectors, so the feature stage does not reduce the dataset size. :contentReference[oaicite:8]index=8

```python
import numpy as np
import scipy.stats as stats

def extract_16_features(noise):
    row_pattern = np.mean(noise, axis=0)

    f1 = np.mean(row_pattern)
    f2 = np.median(row_pattern)
    f3 = np.std(row_pattern)
    f4 = stats.skew(row_pattern)
    f5 = stats.kurtosis(row_pattern)
    f6 = np.min(row_pattern)
    f7 = np.max(row_pattern)
    f8 = np.sum(row_pattern ** 2)

    correlations = []
    row_pattern_std = np.std(row_pattern)

    if row_pattern_std < 1e-8:
        correlations = [0]
    else:
        for row in noise:
            row_std = np.std(row)
            if row_std < 1e-8:
                continue
            corr = np.corrcoef(row, row_pattern)[0, 1]
            if np.isfinite(corr):
                correlations.append(corr)

    if len(correlations) == 0:
        correlations = [0]

    correlations = np.array(correlations)

    f9 = np.mean(correlations)
    f10 = np.median(correlations)
    f11 = np.std(correlations)
    f12 = stats.skew(correlations)
    f13 = stats.kurtosis(correlations)
```

```
    f14 = np.min(correlations)
    f15 = np.max(correlations)
    f16 = np.sum(correlations ** 2)

    features = [f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f12,f13,f14,f15,f16]
    features = np.nan_to_num(features)
    return features
```

**Listing 3:** 16-feature extraction from a residual patch.

### 5.5 Feature preview

The notebook prints out a representative feature vector and shows a dataframe of the 16 values. One visible result is that the row-pattern values are much smaller than the correlation-derived summaries, which is a natural consequence of how the two feature groups are constructed. :contentReference[oaicite:9]index=9

## 6 Classification Setup

### 6.1 Label encoding and scaling

The notebook stores scanner labels as strings and then converts them into integer classes using a label encoder. The feature vectors are standardized before training the SVM, which is especially important because the 16 features come from different statistical scales.

### 6.2 Train-test split with group awareness

A key methodological point is the use of group-based splitting. Instead of randomly mixing patches from the same scanned image into both train and test sets, the notebook keeps image groups together. This matters because patches from the same image are highly correlated. If they were randomly split across train and test, the model could receive an unrealistic advantage due to leakage. The group-aware split used in the notebook avoids that problem. :contentReference[oaicite:10]index=10

### 6.3 Reference SPN features

The notebook also constructs average reference noise patterns per scanner using the training data. These scanner-level reference patterns are then used to build additional correlation features. This is useful because it gives the classifier a class-aware summary of the typical residual structure for each scanner. Importantly, these references are formed only from the training split, which avoids direct leakage into the test set. :contentReference[oaicite:11]index=11

### 6.4 Two-stage grid search

The SVM is tuned in two steps:

1. a coarse grid search over broad values of $C$, $\gamma$, and kernel type,
2. a fine search around the best coarse parameters.

The notebook reports a best coarse configuration around an RBF kernel with $C = 10$ and $\gamma = 0.1$, followed by a fine search that selects the best final model with $C = 50$ and $\gamma = 0.1$. The final test accuracy is 0.8844. This gives a strong balance between model expressiveness and generalization. :contentReference[oaicite:12]index=12

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import GroupShuffleSplit, GroupKFold, GridSearchCV
from sklearn.svm import SVC

le = LabelEncoder()
y_encoded = le.fit_transform(y)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
```

```
train_idx, test_idx = next(gss.split(X_scaled, y_encoded, groups=groups))

X_train, X_test = X_scaled[train_idx], X_scaled[test_idx]
y_train, y_test = y_encoded[train_idx], y_encoded[test_idx]

cv = GroupKFold(n_splits=5)
```

**Listing 4:** SVM training workflow used in the notebook.

# 7 Results

## 7.1 Classification performance

The notebook reports a final test accuracy of 0.8844, or 88.44%. The class-wise report shows strong performance on the two more distinct scanners and comparatively weaker performance on the two HP 6300c units, which is consistent with the fact that those two classes are physically very similar. The macro average is lower than the weighted average because the smaller and more confusable classes reduce the unweighted class mean. :contentReference[oaicite:13]index=13

**Table 2:** Summary of model performance.

| Method | Accuracy |
|---|---|
| 2D correlation baseline | 67.19% |
| 1D row-pattern correlator | 70.31% |
| SVM (RBF kernel) | **88.44%** |

## 7.2 Feature distribution plot

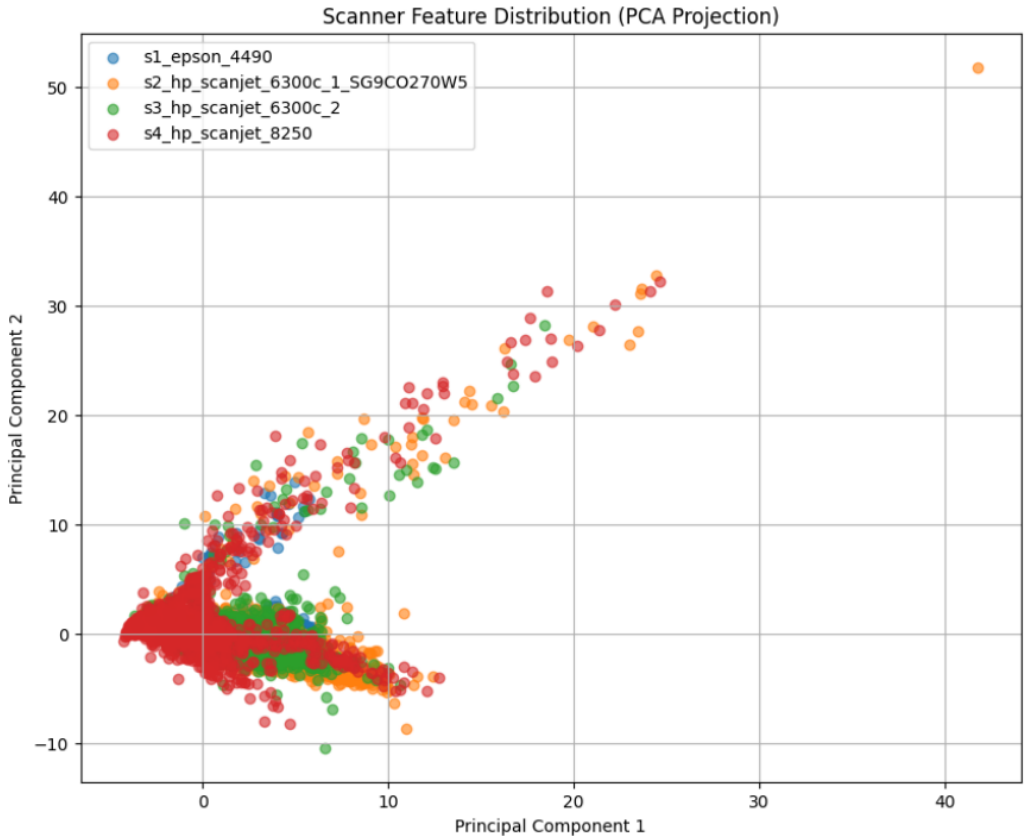



**Figure 5:** Feature distribution after PCA projection onto the first two components.

### 7.3 Confusion matrix

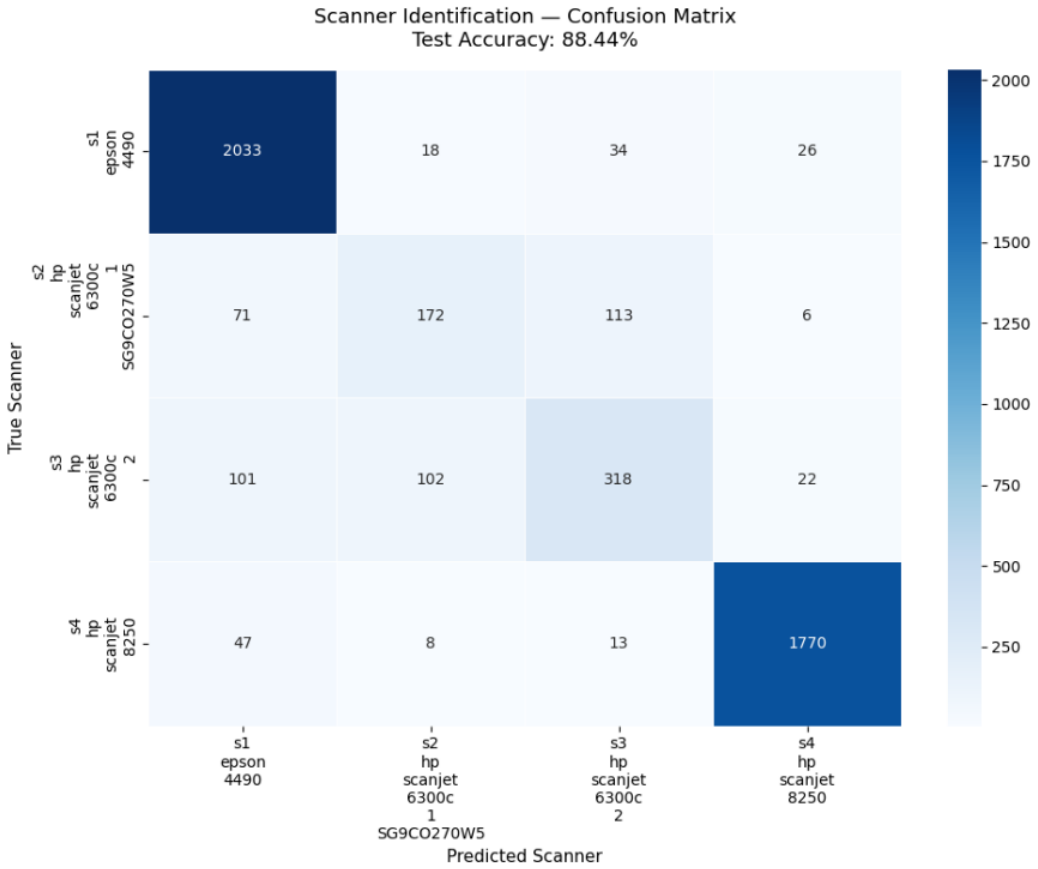



**Figure 6:** Scanner identification confusion matrix.

### 7.4 Interpretation of the results

The classification results show three clear trends. First, the SPN-based feature set is strong enough to discriminate scanner sources with good accuracy. Second, the most difficult errors arise between physically similar devices. Third, a correlation-only baseline is not sufficient on its own, but it becomes useful when combined with the row-pattern statistics and the SVM decision boundary.

## 8 Discussion

### 8.1 Why the method works

The method works because scanner-specific signatures survive the preprocessing stage better than ordinary scene content. The wavelet denoising step suppresses low-level image structure that depends on the scanned document, while the row pattern and correlation features emphasize stable sensor behavior. In effect, the classifier is not looking at what was scanned; it is looking at how the scanner behaved while producing the scan.

### 8.2 Why the HP 6300c units are harder

The notebook results make it clear that the two HP 6300c units are the most difficult pair to separate. This is expected because they belong to the same model family and likely share a very similar internal architecture. Therefore, the residual differences are subtle and require statistical discrimination rather than obvious visual cues. The confusion matrix reflects exactly this difficulty.

### 8.3 Importance of group-aware evaluation

A critical methodological lesson is that patches from the same image must not leak into both training and test splits. If that happened, the classifier would see nearly identical local content during training and testing, which would inflate accuracy. The group-aware split used in the notebook avoids that problem and makes the reported accuracy much more trustworthy.

### 8.4 Limitations

- It depends on high-resolution TIFF scans.
- It is tuned for the available scanner classes.
- The 1200 DPI subset is used for the main experiment, so performance may change on other resolutions.
- The current feature vector is compact and effective, but it may not capture all higher-order residual structure.

### 8.5 Possible improvements

- test more DPI settings,
- add frequency-domain descriptors,
- compare against CNN embeddings,
- evaluate robustness under document content changes,
- analyze cross-device performance on more scanner models.

## 9 Conclusion

This report presented a complete scanner source identification pipeline based on Sensor Pattern Noise and SVM classification. The system begins with a structured TIFF dataset, filters the relevant 1200 DPI images, extracts 24,270 patches, denoises each patch using a 4-level db8 wavelet approach, and converts every residual into a 16-dimensional feature vector. A group-aware train-test split and a two-stage SVM grid search then produce a final accuracy of 88.44%.

The main technical contribution is the combination of a simple preprocessing pipeline with carefully designed statistical features. The main practical conclusion is that scanner fingerprints are strong enough to support reliable source identification when the data is handled carefully and leakage is avoided.

## References


- Python notebook reference supplied for the project workflow and results.
- Wavelet denoising and SPN extraction steps as implemented in the project notebook.
- SVM classification and group-aware evaluation as reported in the project notebook.